

The Multiverse Machine

Development milestones for Claude Code

Nine sequential milestones, in build order. Each one is a working checkpoint, not a pile of loose tasks, so the app is testable end to end after every milestone. Hand these to Claude Code one at a time.

M0 — Project scaffolding

Setup

A clean, running project shell with the chosen stack wired together and deployable from day one.

Tasks

- Initialize React + TypeScript + Vite project
- Install and configure Tailwind CSS
- Set up ESLint/Prettier and a basic folder structure (components, lib, hooks)
- Connect repo to a static host (Vercel or Netlify) with auto-deploy on push
- Add a placeholder page that deploys successfully

Definition of done

- ✓ Empty app builds, deploys, and loads at a public URL.

M1 — In-browser model, proof of branching

Core proof

Prove the central mechanic works before any visual design: load a small model in the browser with Transformers.js and pull real token-by-token alternatives out of it.

Tasks

- Integrate @huggingface/transformers with WebGPU backend
- Load a small instruct model (start with Llama-3.2-1B-Instruct; Qwen2.5-0.5B as fallback)
- Generate text token by token from a hardcoded prompt
- At each step, capture the top-N alternative tokens and their probabilities
- Log the full branching data (chosen token + alternatives + probabilities) to the console

Definition of done

- ✓ Console output shows a real sentence generating with real alternative tokens and probabilities at every step. No UI yet.
-

M2 — Generation engine

Core logic

Turn the proof from M1 into a reusable, controllable engine the UI can drive.

Tasks

- Build a generation module with temperature and branch-count as parameters
- Implement timeline switching: given a chosen alternative token, continue generation from that point
- Structure the output as a tree data model (nodes = tokens, children = alternatives)
- Add loading/progress state for the one-time model download
- Write a minimal plain-HTML test harness to sanity-check outputs without any styling

Definition of done

- ✓ Given a prompt, temperature, and branch count, the engine returns a tree structure that can regenerate from any node, verified in the test harness.

M3 — Branching visualization

Visual core

Render the tree from M2 as the animated, branching, ghost-token visualization that is the heart of the product.

Tasks

- Set up D3.js for an animated node-link tree
- Render the chosen path as a solid, bright line of tokens
- Render alternative tokens as faded ghost branches, opacity/weight scaled by probability
- Animate new tokens appearing live as generation streams in
- Support re-rendering when the user switches to a ghost branch (old path fades back)

Definition of done

- ✓ Typing a prompt produces a live, animated branching tree in the browser matching the approved mockup direction.

M4 — Mac-style app shell

Chrome

Wrap the visualization in the dark, glass, macOS-style window chrome that was approved in the design mockup.

Tasks

- Build the window frame: traffic-light dots, title bar, rounded corners, dark background
- Build the frosted glass control panel component (temperature slider, branch-count control)
- Wire the sliders to the engine from M2 so changes take effect on the next generation
- Style the prompt input and generate button to match the shell
- Responsive pass: tasteful desktop-first layout, clear message on unsupported browsers

Definition of done

- ✓ The full app visually matches the approved Mac-style mockup and the controls actually drive generation.
-

M5 — Interactivity and timeline switching

Interaction

Make the multiverse actually playable: click a ghost, live in that timeline.

Tasks

- Implement click-to-switch on any ghost token, rewinding and continuing from that branch
- Fade the abandoned path visually rather than deleting it
- Add reset (clear canvas) and replay (rewind current tree to the start) controls
- Handle edge cases: switching mid-generation, rapid repeated clicks, empty prompt

Definition of done

- ✓ A user can generate, click any ghost branch, and watch the story convincingly continue from that new reality, repeatedly, without breaking.

M6 — Polish, performance, accessibility

Quality

Take the app from working to genuinely smooth and presentable.

Tasks

- Performance pass: confirm smooth animation and generation speed on a normal laptop
- Add reduced-motion mode for the branching animation
- Verify and improve the WebGPU-unavailable fallback message
- Cross-browser check on Chrome and Edge; confirm the model caches correctly on repeat visits
- Visual QA pass against the approved mockup: spacing, color, type, motion

Definition of done

- ✓ The app feels finished: fast, smooth, accessible, and visually consistent with the approved design.

M7 — Share and export

Growth

Build the feature that turns a play session into a LinkedIn post. This is the growth engine, not an extra.

Tasks

- Implement shareable link: encode prompt + settings + resulting tree in the URL or a saved state
- Reopening a shared link regenerates or replays the same multiverse
- Implement clean image export of the current branching view for sharing
- Add a visible privacy line in the UI: nothing typed ever leaves the browser

Definition of done

- ✓ A user can generate a multiverse, export a polished image of it, and share a link that reopens the same result.
-

M8 — Launch prep

Ship

Final hardening and a clean public launch.

Tasks

- Full walkthrough test: first-time visitor, slow connection, repeat visitor
- Add a minimal about/how-it-works line in the UI for first-time context
- Confirm static hosting handles expected traffic at zero ongoing cost
- Prepare launch assets: a few example prompts, exported images for the announcement post

Definition of done

- ✓ The app is live at a public URL, tested cold, and ready to be posted.
-